

Comprehensive Report on Real-Time Speech Transcription Systems

This report covers three implementations of real-time speech transcription systems. Each system leverages different tools and methodologies to achieve the same goal: capturing audio input, transcribing it to text, and optionally classifying the transcribed content. The three implementations focus on Python-based solutions using Hugging Face Whisper, PyTorch, and C++-based approaches.

1. Python Implementation with Hugging Face Whisper

Description:

This implementation uses the Hugging Face `transformers` library to integrate the Whisper model for real-time speech transcription. It captures audio using `PyAudio`, processes it using the Whisper model, and logs the transcriptions in a JSON file.

Key Features:

1. Real-Time Audio Processing:

- Utilizes `pyaudio` to capture audio in chunks.
- Processes chunks in real-time to maintain low latency.

2. Whisper Model:

- Employs Hugging Face's Whisper model for transcription.
- Capable of recognizing multiple languages (with model variations).

3. JSON Logging:

- Each transcription segment is logged with start and end times.
- Maintains a JSON file that is cleared when its size exceeds 50MB to ensure performance and storage efficiency.

4. Classification:

- Classifies text segments into predefined categories such as `bemo`, `music`, `screaming`, etc.

2. Python Implementation with PyTorch and Whisper

Description:

This implementation directly integrates OpenAI's Whisper model using PyTorch. It transcribes WAV files or real-time audio, processes the results, and classifies the text using a custom function.

Key Features:

1. WAV File Transcription:

- Processes pre-recorded audio files in WAV format.

2. Classification:

- Identifies categories (e.g., `bemo`, `music`, `screaming`) from transcribed text.

3. Real-Time Transcription:

- Supports real-time audio transcription using `speech\_recognition` and Whisper.

4. JSON Logging:

- Logs transcription details into a JSON file.

3. C++ Implementation with Whisper.cpp

Description:

This implementation uses `whisper.cpp`, a high-performance C++ port of the Whisper model. It is designed for low-latency transcription on edge devices.

Key Features:

1. Real-Time Transcription:

- Captures audio input using `PortAudio`.
- Processes audio in chunks to provide immediate transcription results.

2. Classification:

- Classifies transcriptions into categories such as `bemo`, `music`, etc., using a function that checks for specific keywords in the text.

3. JSON Logging:

- Logs transcription data (raw text, cleaned text, classification, start and end times) into a JSON file.

4. File Management:

- Includes a function to clear the JSON file when it exceeds 50MB.

Comparison of the Three Implementations

Feature	Hugging Face Whisper (Python)	PyTorch Whisper (Python)	Whisper.cpp (C++)
Programming Language	Python	Python	C++
Audio Input	Real-time	Real-time & WAV files	Real-time
Transcription Model	Hugging Face Whisper	PyTorch Whisper	Whisper.cpp

Classification	Yes	Yes	Yes
JSON Logging	Yes	Yes	Yes
Performance	Moderate	Moderate	High
Ease of Integration	High	High	Moderate
File Management	JSON file cleared at 50MB	JSON file logging	JSON file cleared at 50MB

Conclusion and Recommendations

1. Use Case Selection:

- **Hugging Face Whisper**: Best suited for projects requiring easy integration, rapid development, and flexible deployment.
- **PyTorch Whisper**: Ideal for applications needing advanced control over input types and GPU acceleration.
- **Whisper.cpp**: Recommended for resource-constrained environments or high-performance requirements.

2. Future Enhancements:

- Add speaker identification for multi-speaker environments.
- Enhance classification with natural language processing (NLP) techniques.
- Integrate real-time visual displays for live transcription monitoring.

These implementations demonstrate the versatility and power of Whisper in real-time speech transcription and classification, providing robust solutions for various applications.